# 3.1、色彩空间转换

[OpenCV: Changing Colorspaces]
https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html

颜色空间转换是图像处理和计算机视觉中最基础、最常用的操作之一，尤其在颜色检测、物体跟踪、皮肤检测等任务中几乎必用

# 1. 核心函数：cv.cvtColor()

```
dst = cv.cvtColor(src, code)
```

- src：输入图像
- code：转换标志，例如：
  - cv.COLOR_BGR2GRAY
  - cv.COLOR_BGR2HSV
  - cv.COLOR_HSV2BGR
  - cv.COLOR_BGR2RGB 等
    想查看所有支持的标志，可以运行：

```
import cv2 as cv
flags = [i for i in dir(cv) if i.startswith('COLOR_')]
print(flags)   # 会打印非常长的列表
```

# 2. 重要颜色空间对比

| 颜色空间 | 通道顺序 | 特点 | 主要用途 |
|---|---|---|---|
| BGR | Blue, Green, Red | OpenCV 默认读取图像的格式 | 图像显示、保存 |
| RGB | Red, Green, Blue | 符合人类直觉，其他库常用 | 与 Matplotlib 等配合 |
| Gray | 单通道灰度值 | 0~255，亮度信息 | 减少计算量、边缘检测等 |
| HSV | Hue(色调), Saturation(饱和度), Value(亮度) | 把颜色（Hue）与亮度分开 | 颜色检测、物体跟踪（最常用！） |

HSV 特别重要：

- Hue（色调）：0~179（注意！不是 0~360）
- Saturation（饱和度）：0~255
- Value（亮度）：0~255
  不同软件 HSV 范围不同，用 OpenCV 时一定要注意 Hue 只到 179。

# 3. 实际代码示例

示例1：图像颜色空间转换

```
import cv2 as cv
import numpy as np

img = cv.imread('messi5.jpg')          # 替换成你的图片
assert img is not None, "图片读取失败"

# BGR → Gray
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

# BGR → HSV
hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)

# HSV → BGR（转换回来）
bgr_back = cv.cvtColor(hsv, cv.COLOR_HSV2BGR)

cv.imshow('Original BGR', img)
cv.imshow('Gray', gray)
cv.imshow('HSV', hsv)
cv.imshow('Back to BGR', bgr_back)
cv.waitKey(0)
cv.destroyAllWindows()
```

示例2：如何找到某种颜色的 HSV 值（非常实用！）

```
# 想知道纯绿色的 HSV 值
green = np.uint8([[[0, 255, 0]]])        # BGR 格式的纯绿色
hsv_green = cv.cvtColor(green, cv.COLOR_BGR2HSV)
print(hsv_green)                         # 输出 [[[ 60 255 255]]]
```

提取绿色物体的范围建议：

- lower = [50, 100, 100]
- upper = [70, 255, 255]
  （Hue ±10，Saturation 和 Value 保持较高）你可以用同样的方法测试红色、蓝色、黄色等颜

色。

## 4. 提取特定颜色 + 视频实时跟踪（教程核心应用）

完整代码（跟踪蓝色物体）：

```
import cv2 as cv
import numpy as np

cap = cv.VideoCapture(0)   # 0 表示默认摄像头

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # 1. 转换到 HSV
    hsv = cv.cvtColor(frame, cv.COLOR_BGR2HSV)

    # 2. 设置蓝色在 HSV 中的范围（可根据实际调整）
    lower_blue = np.array([110, 50, 50])
    upper_blue = np.array([130, 255, 255])

    # 3. 创建掩码（mask）：在范围内的像素为 255，其余为 0
    mask = cv.inRange(hsv, lower_blue, upper_blue)

    # 4. 用掩码提取原图中的蓝色部分
    res = cv.bitwise_and(frame, frame, mask=mask)

    # 显示三个窗口
    cv.imshow('Original Frame', frame)
    cv.imshow('Mask', mask)
    cv.imshow('Result (Only Blue)', res)

    k = cv.waitKey(5) & 0xFF
    if k == 27:        # 按 ESC 键退出
        break

cap.release()
cv.destroyAllWindows()
```

运行效果：

- Original：原始彩色画面
- Mask：二值图像，只显示蓝色区域为白色

- Result：只保留蓝色的彩色图像